YIELD METHOD

22 February 2025 11:30

- Yield method stops the current executing thread and give chance to other thread for execution
- The yield() method in Java is a part of Thread class and is used in multithreading to signal the CPU that the current thread is willing to give up its CPU time for other threads of the same or higher priority.

Real-Life Shopping Example Using yield() in Java 🛒
Scenario: Customers at a Billing Counter in a Mall 💳
Imagine a shopping mall where multiple customers are standing in line at the billing counter to pay for their purchases.

- Some customers have fewer items and should be given priority.
- A customer with a full cart may yield (let another customer go first) if they notice a shorter queue behind them.
- The cashier (CPU) decides who gets the turn, but yield() allows customers to let others go ahead if needed.

1. yield() Method Definition

public static native void yield();

- It is static and native (implemented in diff language , not Java).
- Belongs to the Thread class.
- It does not guarantee that another thread will get the CPU.
- Only suggests to the JVM that it can switch to another thread.

2. How yield() Works?

- Thread provide hints to thread schedular then it depend on thread schedular to accept or ignore the hint.
- Output may be different because it depend upon thread schedular.
- When a thread calls Thread.yield(), the CPU scheduler may pause the current thread.
- The thread goes back to the READY state (not BLOCKED or WAITING).
- The CPU may continue with the same thread if no other threads are ready.

When to Use yield()?

- Use yield() when:

1. You want to give other threads a chance to execute.
2. You have multiple threads of the same priority.
3. You want to improve responsiveness in a CPU-intensive application.

- Do NOT use yield() if:

1. You want guaranteed thread switching (use sleep() instead).
2. Your program logic depends on a thread yielding (it's unpredictable).
3. You need proper thread synchronization (use wait() and notify() instead).

```
class Customer extends Thread {
    private String customerName;
    private int items;

    public Customer(String customerName, int items) {
        this.customerName = customerName;
        this.items = items;
    }

    public void run() {
        System.out.println(customerName + " has " + items + " items and is at the counter.");

        // If a customer has more than 5 items, they may let others go first
        if (items > 5) {
            System.out.println(customerName + " has too many items, allowing another customer first...");
            Thread.yield(); // Suggests the CPU to give another customer a turn
        }

        System.out.println(customerName + " is paying for items.");
    }
}

public class ShoppingMall {
    public static void main(String[] args) {
        Customer c1 = new Customer("Alice", 3);  // Small cart
        Customer c2 = new Customer("Bob", 8);    // Large cart
        Customer c3 = new Customer("Charlie", 2); // Small cart
```

```
        c1.start();
        c2.start();
        c3.start();
    }
}
```

Conclusion

- yield() is a useful hint to the CPU for thread switching.
- It does not guarantee execution transfer.
- It does not release locks and should not be relied upon for thread synchronization.
- In modern applications, Thread.sleep() and proper synchronization are preferred over yield().

Common Questions & Answers

Q1: What if no other thread is ready when yield() is called?

- The CPU may continue with the same thread.

Q2: Will yield() always switch to another thread?

- No, it only suggests to the scheduler.

Q3: Can yield() cause performance issues?

- Yes, excessive use may lead to unnecessary context switching, reducing performance.

Q4: Is yield() commonly used in real projects?

- Rarely. Instead, developers use Thread.sleep() or proper synchronization mechanisms.

Q5: What is the difference between yield() and Thread Priority?

- yield() is a method that suggests CPU switching.
- Thread Priority is a property that influences the scheduling algorithm.